EXP 5: To write C program to implement SJF CPU scheduling algorithm

a) SJF (Non-primptive mode)

```
lab-03-17@lab-03-17-OptiPlex-3280-AIO: ~
lab-03-17@lab-03-17-OptiPlex-3280-AIO:~$ vim sjf_np_1290.c
lab-03-17@lab-03-17-OptiPlex-3280-AIO:~$ gcc sjf_np_1290.c
lab-03-17@lab-03-17-OptiPlex-3280-AIO:~$ ./a.out
Enter number of processes: 4
Enter Arrival Time and Burst Time for P1: 0
3
Enter Arrival Time and Burst Time for P2: 1
3
Enter Arrival Time and Burst Time for P3: 5
2
Enter Arrival Time and Burst Time for P4: 4
2

P       AT      BT      CT      WT      TAT
P1      0       3       3       0       3
P2      1       3       6       2       5
P3      5       2       8       1       3
P4      4       2       10      4       6

Average Waiting Time = 1.75
Average Turnaround Time = 4.25
Average Completion Time = 6.75
Throughput = 0.40 processes/unit time


Gantt Chart:

 --------------------------------
|  P1   |  P2   |  P3   |  P4   |
 --------------------------------
0       3       6       8       10lab-03-17@lab-03-17-OptiPlex-3280-AIO:~$
```

```
lab-03-17@lab-03-17-OptiPlex-3280-AIO: ~
#include <stdio.h>

int main() {
    int n, i;
    int at[10], bt[10], ct[10], wt[10], tat[10], p[10];
    int completed[10] = {0};
    int time = 0, count = 0, shortest;
    float avg_wt = 0, avg_tat = 0, avg_ct = 0, throughput;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        p[i] = i + 1;
        printf("Enter Arrival Time and Burst Time for P%d: ", p[i]);
        scanf("%d %d", &at[i], &bt[i]);
    }

    // SJF Scheduling
    while (count < n) {
        shortest = -1;
        int min_bt = 9999;

        for (i = 0; i < n; i++) {
            if (at[i] <= time && completed[i] == 0) {
                if (bt[i] < min_bt) {
                    min_bt = bt[i];
                    shortest = i;
                }
            }
        }

        if (shortest == -1) {
            time++;   // If no process has arrived
        } else {
            time += bt[shortest];
            ct[shortest] = time;
            tat[shortest] = ct[shortest] - at[shortest];
            wt[shortest] = tat[shortest] - bt[shortest];

            avg_wt += wt[shortest];
            avg_tat += tat[shortest];
            avg_ct += ct[shortest];

            completed[shortest] = 1;
            count++;
```

lab-03-17@lab-03-17-OptiPlex-3280-AIO: ~

```
            count++;
        }
    }

    // Calculate averages
    avg_wt /= n;
    avg_tat /= n;
    avg_ct /= n;

    throughput = (float)n / (time - at[0]);

    printf("\nP\tAT\tBT\tCT\tWT\tTAT\n");
    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
               p[i], at[i], bt[i], ct[i], wt[i], tat[i]);
    }

    printf("\nAverage Waiting Time = %.2f", avg_wt);
    printf("\nAverage Turnaround Time = %.2f", avg_tat);
    printf("\nAverage Completion Time = %.2f", avg_ct);
    printf("\nThroughput = %.2f processes/unit time\n", throughput);

    // Gantt Chart
    printf("\n\nGantt Chart:\n\n");

    printf(" ");
    for (i = 0; i < n; i++) {
        printf("--------");
    }
    printf("\n|");

    // Print in execution order
    int printed[10] = {0};
    int current_time = 0;

    for (int k = 0; k < n; k++) {
        int min_ct = 9999, pos = -1;
        for (i = 0; i < n; i++) {
            if (!printed[i] && ct[i] < min_ct) {
                min_ct = ct[i];
                pos = i;
            }
        }
        printf("  P%d   |", p[pos]);
        printed[pos] = 1;
    }
```

b) SJF (Primptive Mode – SRTF)

```
lab-03-17@lab-03-17-OptiPlex-3280-AIO: $ vim sjf_p_1290.c
lab-03-17@lab-03-17-OptiPlex-3280-AIO: $ gcc sjf_p_1290.c
lab-03-17@lab-03-17-OptiPlex-3280-AIO: $ ./a.out
Enter number of processes: 4
Enter Arrival Time and Burst Time for P1: 0
4
Enter Arrival Time and Burst Time for P2: 2
4
Enter Arrival Time and Burst Time for P3: 1
4
Enter Arrival Time and Burst Time for P4: 2
7

P       AT      BT      CT      WT      TAT
P1      0       4       4       0       4
P2      2       4       8       2       6
P3      1       4       12      7       11
P4      2       7       19      10      17

Average Waiting Time = 4.75
Average Turnaround Time = 9.50
Average Completion Time = 10.75
Throughput = 0.21 processes/unit time


Gantt Chart:

| P1 | P1 | P1 | P1 | P2 | P2 | P2 | P2 | P3 | P3 | P3 | P3 | P4 | P4 | P4 | P4 | P4 | P4 | P4 |
0  1  2  3  4  5  6  7  8  9  10  11  12  13  14  15  16  17  18  19lab-03-17@lab-03-17-OptiPlex-3280-AIO: $
```

lab-03-17@lab-03-17-OptiPlex-3280-AIO: ~

```c
#include <stdio.h>

int main() {
    int n, i;
    int at[10], bt[10], rt[10], ct[10], wt[10], tat[10], p[10];
    int time = 0, count = 0, shortest;
    float avg_wt = 0, avg_tat = 0, avg_ct = 0, throughput;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        p[i] = i + 1;
        printf("Enter Arrival Time and Burst Time for P%d: ", p[i]);
        scanf("%d %d", &at[i], &bt[i]);
        rt[i] = bt[i];   // Remaining time initially = burst time
    }

    int gantt[100], g_index = 0;

    // SRTF Scheduling
    while (count < n) {
        shortest = -1;
        int min_rt = 9999;

        for (i = 0; i < n; i++) {
            if (at[i] <= time && rt[i] > 0) {
                if (rt[i] < min_rt) {
                    min_rt = rt[i];
                    shortest = i;
                }
            }
        }

        if (shortest == -1) {
            time++;
        } else {
            gantt[g_index++] = p[shortest];
            rt[shortest]--;   // Execute for 1 unit
            time++;

            if (rt[shortest] == 0) {
                count++;
                ct[shortest] = time;
                tat[shortest] = ct[shortest] - at[shortest];
                wt[shortest] = tat[shortest] - bt[shortest];
```

lab-03-17@lab-03-17-OptiPlex-3280-AIO: ~

```c
            count++;
        }
    }

    // Calculate averages
    avg_wt /= n;
    avg_tat /= n;
    avg_ct /= n;

    throughput = (float)n / (time - at[0]);

    printf("\nP\tAT\tBT\tCT\tWT\tTAT\n");
    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
               p[i], at[i], bt[i], ct[i], wt[i], tat[i]);
    }

    printf("\nAverage Waiting Time = %.2f", avg_wt);
    printf("\nAverage Turnaround Time = %.2f", avg_tat);
    printf("\nAverage Completion Time = %.2f", avg_ct);
    printf("\nThroughput = %.2f processes/unit time\n", throughput);

    // Gantt Chart
    printf("\n\nGantt Chart:\n\n");

    printf(" ");
    for (i = 0; i < n; i++) {
        printf("--------");
    }
    printf("\n|");

    // Print in execution order
    int printed[10] = {0};
    int current_time = 0;

    for (int k = 0; k < n; k++) {
        int min_ct = 9999, pos = -1;
        for (i = 0; i < n; i++) {
            if (!printed[i] && ct[i] < min_ct) {
                min_ct = ct[i];
                pos = i;
            }
        }
        printf("  P%d   |", p[pos]);
        printed[pos] = 1;
    }
```

lab-03-17@lab-03-17-OptiPlex-3280-AIO: ~

```
            if (rt[shortest] == 0) {
                count++;
                ct[shortest] = time;
                tat[shortest] = ct[shortest] - at[shortest];
                wt[shortest] = tat[shortest] - bt[shortest];

                avg_wt += wt[shortest];
                avg_tat += tat[shortest];
                avg_ct += ct[shortest];
            }
        }
    }

    // Calculate averages
    avg_wt /= n;
    avg_tat /= n;
    avg_ct /= n;
    throughput = (float)n / (time - at[0]);

    printf("\nP\tAT\tBT\tCT\tWT\tTAT\n");
    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
               p[i], at[i], bt[i], ct[i], wt[i], tat[i]);
    }

    printf("\nAverage Waiting Time = %.2f", avg_wt);
    printf("\nAverage Turnaround Time = %.2f", avg_tat);
    printf("\nAverage Completion Time = %.2f", avg_ct);
    printf("\nThroughput = %.2f processes/unit time\n", throughput);

    // Gantt Chart
    printf("\n\nGantt Chart:\n\n|");

    for (i = 0; i < g_index; i++) {
        printf(" P%d |", gantt[i]);
    }

    printf("\n0");
    for (i = 1; i <= g_index; i++) {
        printf("  %d", i);
    }

    return 0;
}
```